

Experiment No 8

Aim :

To understand and implement basic firewall rule-based access control by blocking and allowing ports, logging traffic, and testing connectivity using simple tools such as Windows Defender Firewall (Windows) or iptables (Linux VM). As we have a Windows Host OS with a Linux VM,

we will use your Windows machine as the "Tester" and your Linux VM as the "Defender".

Lab Topology & Setup

- Machine A (The Defender): Your Linux VM.
- Machine B (The Tester): Your Windows Host OS.
- Network Requirement: Your Linux VM network adapter must be set to Bridged or Host-Only in your hypervisor (VirtualBox/VMware) so your Windows machine can communicate with it.

Phase 0: Baseline Connectivity Testing

Before implementing any firewall rules, we must prove the network is open and the two machines can talk.

1. **On your Linux VM**, open a terminal and find its IP address:

`ip a`

(Look for the inet address under your main network interface, e.g., 192.168.x.x)

2. **On your Windows Host**, open Command Prompt (cmd) and ping the Linux VM:

DOS

`ping <LINUX_IP_ADDRESS>`

Result: You should see successful replies. Leave this Command Prompt open.

Phase 1: Blocking ICMP (Ping) & Enabling Logging

In iptables, rules are read from top to bottom. If we want to log a dropped packet, we must tell the firewall to log it *before* we tell it to drop it.

1. **On your Linux VM**, add a rule to **LOG** incoming ping requests. We will add a custom prefix so the logs are easy to find:

`sudo iptables -A INPUT -p icmp --icmp-type echo-request -j LOG --log-prefix "FW_PING_DROP: "`

(Note: -A means "Append" to the bottom of the list).

2. **On your Linux VM**, add a rule to **DROP** the incoming ping requests:

```
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
```

3. **Test the Block:** Go back to your Windows Command Prompt and ping the Linux VM again:

DOS

```
ping <LINUX_IP_ADDRESS>
```

Result: The request will now time out. Your firewall is actively dropping the packets.

Phase 2: Viewing the Firewall Logs

Because we set up the logging rule in Phase 1, the Linux kernel secretly recorded every time your Windows machine tried to ping it.

1. **On your Linux VM**, search your system logs for the custom prefix we created.
 - If you are on Ubuntu/Debian/Kali:

```
sudo grep "FW_PING_DROP" /var/log/syslog
```

- If you are on CentOS/RHEL/Fedora:

```
sudo grep "FW_PING_DROP" /var/log/messages
```

2. *(Alternative command if the above files are restricted: `sudo dmesg | grep "FW_PING_DROP"`)*
3. **Analyze the Log:** You will see an output containing the SRC (Your Windows IP), DST (Your Linux IP), and the MAC addresses of the blocked connection attempt.

Phase 3: Rule Precedence (Blocking and Allowing a Port)

To truly understand firewalls, you need to see how rule order affects traffic. We will set up a temporary web server, block it, and then punch a hole through the block.

Step A: Set up the Service

1. **On your Linux VM**, start a temporary Python web server on port 8000:

```
python3 -m http.server 8000
```

2. **On your Windows Host**, open a web browser and go to `http://<LINUX_IP_ADDRESS>:8000`. You should see a directory listing.
3. *Stop the Python server on the VM by pressing `Ctrl+C` so we can type more commands.*

Step B: Block the Port

1. **On your Linux VM**, append a rule to drop all TCP traffic heading to port 8000:

```
sudo iptables -A INPUT -p tcp --dport 8000 -j DROP
```

2. Start the Python server again (python3 -m http.server 8000).
3. Refresh the web browser on your Windows host. *It will hang and fail to load.*

Step C: Override the Block (The Power of Rule Order) Right now, the DROP rule is at the bottom of the chain. If we *Append* (-A) an ACCEPT rule, it will go below the DROP rule, and the firewall will never read it. We must *Insert* (-I) the ACCEPT rule at the top of the chain so it is read first.

1. Stop the Python server (Ctrl+C).
2. **On your Linux VM**, **Insert** an ACCEPT rule at position 1 (the very top):

```
sudo iptables -I INPUT 1 -p tcp --dport 8000 -j ACCEPT
```

3. View your current rules list to verify the order:

```
sudo iptables -L --line-numbers
```

(You should see your ACCEPT rule at line 1, and your DROP rule further down).

4. Start the Python server one last time (python3 -m http.server 8000).
5. Refresh your Windows browser. *It works again!* The firewall read the ACCEPT rule first, allowed the traffic, and ignored the DROP rule below it.

Phase 4: Cleanup

It is important to reset your VM's firewall so you don't accidentally block yourself from future labs.

1. Stop the Python server (Ctrl+C).
2. **On your Linux VM**, flush (delete) all custom iptables rules:

```
sudo iptables -F
```

3. Verify the rules are gone:

```
sudo iptables -L
```

Conclusion :

You have successfully implemented rule-based access control, logged malicious traffic, and manipulated firewall rule precedence.

